



Introduction to Machine Learning (CS 436/536)

Lecture-4: PyTorch Basics
arakin@binghamton.edu

✓
0s

```
[1] # importing the numpy Module  
import numpy as np
```

✓
0s

```
[2] # importing the Torch Module  
import torch
```

✓
0s

```
[3] # Defining a Tensor  
x = torch.arange(12)  
x  
  
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

✓
0s

```
[4] # Shape of the tensor  
x.shape  
  
torch.Size([12])
```

✓
0s

```
[5] # Number of element in X  
x.numel()  
  
12
```

✓
0s

```
[6] # Anytime you can reshape a tensor  
X = x.reshape(3, 4)  
X  
  
tensor([[ 0,  1,  2,  3],  
        [ 4,  5,  6,  7],  
        [ 8,  9, 10, 11]])
```

✓
0s



Defining tensors with zeros

```
torch.zeros((2, 3, 4))
```

```
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],
        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]])
```

✓
0s

[8] # Defining tensors with ones

```
torch.ones((2, 3, 4))
```

```
tensor([[[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]],
        [[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]])
```

✓
0s

[9] # defining tensors with random weight

```
torch.randn(3, 4)
```

```
tensor([[-0.2672, -0.1319,  0.3272,  1.0032],
        [-0.5702,  0.5869, -0.2229, -0.4348],
        [-0.1092,  0.6014,  0.0545,  1.1707]])
```

✓
0s

```
[10] # Manually defining tensors
      print(torch.tensor([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]]))

      tensor([[2, 1, 4, 3],
              [1, 2, 3, 4],
              [4, 3, 2, 1]])
```

✓
0s

```
[11] # operations
      x = torch.tensor([1.0, 2, 4, 8])
      y = torch.tensor([2, 2, 2, 2])
      x + y, x - y, x * y, x / y, x**y # The ** operator is exponentiation

      (tensor([ 3.,  4.,  6., 10.]),
       tensor([-1.,  0.,  2.,  6.]),
       tensor([ 2.,  4.,  8., 16.]),
       tensor([0.5000, 1.0000, 2.0000, 4.0000]),
       tensor([ 1.,  4., 16., 64.]))
```

✓
0s

```
[12] torch.exp(x)

      tensor([2.7183e+00, 7.3891e+00, 5.4598e+01, 2.9810e+03])
```

✓
0s

```
[13] # Joining two tensor across different dimensions
      X = torch.arange(12, dtype=torch.float32).reshape((3, 4))
      Y = torch.tensor([[2.0, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])
      torch.cat((X, Y), dim=0), torch.cat((X, Y), dim=1)

      (tensor([[ 0.,  1.,  2.,  3.],
                [ 4.,  5.,  6.,  7.],
                [ 8.,  9., 10., 11.],
                [ 2.,  1.,  4.,  3.],
                [ 1.,  2.,  3.,  4.],
                [ 4.,  3.,  2.,  1.]]),
       tensor([[ 0.,  1.,  2.,  3.,  2.,  1.,  4.,  3.],
                [ 4.,  5.,  6.,  7.,  1.,  2.,  3.,  4.],
                [ 8.,  9., 10., 11.,  4.,  3.,  2.,  1.])))
```

✓ [14] # logical operations

0s

```
X == Y
```

```
tensor([[False,  True, False,  True],
        [False, False, False, False],
        [False, False, False, False]])
```

✓ [15] # dot operations

0s

```
X.sum(), X.abs(), X.mean()
```

```
(tensor(66.), tensor([[ 0.,  1.,  2.,  3.],
                     [ 4.,  5.,  6.,  7.],
                     [ 8.,  9., 10., 11.]]), tensor(5.5000))
```

✓ [16] a = torch.arange(3).reshape((3, 1))

0s

```
b = torch.arange(2).reshape((1, 2))
```

```
a, b, a+b
```

```
(tensor([[0],
         [1],
         [2]]), tensor([[0, 1]]), tensor([[0, 1],
         [1, 2],
         [2, 3]]))
```

✓ [17] # indexing and slicing

0s

```
X, X[-1], X[1:3]
```

```
(tensor([[ 0.,  1.,  2.,  3.],
         [ 4.,  5.,  6.,  7.],
         [ 8.,  9., 10., 11.]]),
 tensor([ 8.,  9., 10., 11.]),
 tensor([[ 4.,  5.,  6.,  7.],
         [ 8.,  9., 10., 11.])))
```

✓ [18] # Manipulating values

0s

```
X[1, 2] = 9
```

```
X
```

```
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  9.,  7.],
        [ 8.,  9., 10., 11.]])
```

✓ [19] X[0:2, :] = 12

0s

```
X
```

```
tensor([[12., 12., 12., 12.],
        [12., 12., 12., 12.],
        [ 8.,  9., 10., 11.]])
```

✓ [20] # Memory

0s

```
before = id(Y)
```

```
Y = Y + X
```

```
id(Y) == before
```

```
False
```

✓ [21] Z = torch.zeros_like(Y)

0s

```
print('id(Z):', id(Z))
```

```
Z[:] = X + Y
```

```
print('id(Z):', id(Z))
```

```
id(Z): 140096993665584
```

```
id(Z): 140096993665584
```

✓
0s

```
▶ before = id(X)  
X += Y  
id(X) == before
```

☞ True

✓
0s

[23] # Conversions to other python scripts

```
A = X.numpy()  
B = torch.from_numpy(A)  
type(A), type(B)
```

(numpy.ndarray, torch.Tensor)

✓
1s

[24] a = torch.tensor([3.5])
a, a.item(), float(a), int(a)

(tensor([3.5000]), 3.5, 3.5, 3)

✓
0s

[25] # Linear Algebra

```
x = torch.tensor(3.0)  
y = torch.tensor(2.0)  
x + y, x * y, x / y, x**y
```

(tensor(5.), tensor(6.), tensor(1.5000), tensor(9.))

✓ [26] # vectors
0s x = torch.arange(4)
x

tensor([0, 1, 2, 3])

✓ [27] len(x)
15s

4

✓ [28] #Matrix Transpose
0s A = torch.arange(20).reshape(5, 4)
A,A.T

(tensor([[0, 1, 2, 3],
[4, 5, 6, 7],
[8, 9, 10, 11],
[12, 13, 14, 15],
[16, 17, 18, 19]]), tensor([[0, 4, 8, 12, 16],
[1, 5, 9, 13, 17],
[2, 6, 10, 14, 18],
[3, 7, 11, 15, 19]]))

✓ [29] # Tensor operations
0s A = torch.arange(20, dtype=torch.float32).reshape(5, 4)
B = A.clone() # Assign a copy of `A` to `B` by allocating new memory
A, A + B

(tensor([[0., 1., 2., 3.],
[4., 5., 6., 7.],
[8., 9., 10., 11.],
[12., 13., 14., 15.],
[16., 17., 18., 19.]]), tensor([[0., 2., 4., 6.],
[8., 10., 12., 14.],
[16., 18., 20., 22.],
[24., 26., 28., 30.],
[32., 34., 36., 38.]])

✓
0s



A * B

```
↳ tensor([[ 0.,  1.,  4.,  9.],
          [16., 25., 36., 49.],
          [64., 81., 100., 121.],
          [144., 169., 196., 225.],
          [256., 289., 324., 361.]])
```

✓
0s

[31] # sum operation

```
x = torch.arange(4, dtype=torch.float32)
x, x.sum()
```

```
(tensor([0., 1., 2., 3.]), tensor(6.))
```

✓
0s

[32] A.shape, A.sum()

```
(torch.Size([5, 4]), tensor(190.))
```

✓
0s

[33] # reduction along one direction

```
A_sum_axis0 = A.sum(axis=0)
A_sum_axis0, A_sum_axis0.shape
```

```
(tensor([40., 45., 50., 55.]), torch.Size([4]))
```

✓
0s

[34] A_sum_axis1 = A.sum(axis=1)

```
A_sum_axis1, A_sum_axis1.shape
```

```
(tensor([ 6., 22., 38., 54., 70.]), torch.Size([5]))
```

✓
0s

```
[35] # dot product
y = torch.ones(4, dtype=torch.float32)
x, y, torch.dot(x, y)
```

```
(tensor([0., 1., 2., 3.]), tensor([1., 1., 1., 1.]), tensor(6.))
```

✓
0s

```
[36] # Alternative
torch.sum(x * y)
```

```
tensor(6.)
```

✓
0s

```
[37] # multiplication
A.shape, x.shape, torch.mv(A, x)
```

```
(torch.Size([5, 4]), torch.Size([4]), tensor([ 14.,  38.,  62.,  86., 110.]))
```

✓
0s

```
[38] # Same
B = torch.ones(4, 3)
torch.mm(A, B)
```

```
tensor([[ 6.,  6.,  6.],
        [22., 22., 22.],
        [38., 38., 38.],
        [54., 54., 54.],
        [70., 70., 70.]])
```

✓
0s

```
[39] # Norm Calculation
torch.norm(torch.ones((4, 9)))
```

```
tensor(6.)
```

✓ [40] # calculus: Differentiation
0s x = torch.arange(4.0)
x

tensor([0., 1., 2., 3.])

✓ [41] x = torch.arange(4.0)
0s x

tensor([0., 1., 2., 3.])

✓ [42] x.requires_grad_(True) # Same as `x = torch.arange(4.0, requires\_grad=True)`
0s print(x.grad) # The default value is None

None

✓ [43] # forward operation
0s y = 2 * torch.dot(x, x)
y

tensor(28., grad_fn=<MulBackward0>)

✓ [44] y.backward()
0s x.grad # double check this using hand calculation

tensor([0., 4., 8., 12.])

✓ [45] # result
0s x.grad == 4 * x

tensor([True, True, True, True])

✓ 0s  # PyTorch accumulates the gradient in default, we need to clear the previous values

```
x.grad.zero_()
y = x.sum()
y.backward()
x.grad
```

 tensor([1., 1., 1., 1.])

✓ 0s [47] # Detach removes the need for keeping records of gradient

```
x.grad.zero_()
y = x * x
u = y.detach()
z = u * x
z.sum().backward()
print(u.grad)
```

None

✓ 0s [48] # Pytorch Loss functions

```
loss = torch.nn.MSELoss
# more loss functions : https://pytorch.org/docs/stable/nn.html#loss-functions
```

✓ 0s [49] # Pytorch optimizer

```
optimizer = torch.optim.SGD([X], lr=0.01, momentum=0.9)
optimizer.step()
# more optimizers: https://pytorch.org/docs/stable/optim.html
```

✓
0s



An working Example

```
inputs = torch.tensor([2],dtype = float) # input
targets = torch.tensor([1],dtype = float) # target output
Inter_var = torch.tensor([0.25]) # variable that changes the input : output = input * Inter_var
Inter_var.requires_grad = True # updating this variable using optimizer
outputs = inputs * Inter_var
optimizer = torch.optim.SGD([Inter_var], lr=0.05)
loss = torch.nn.MSELoss(reduction = 'mean') # loss function complete definition
loss_comp = loss (outputs,targets) # loss compute
print(outputs,loss_comp)
```

tensor([0.5000], dtype=torch.float64, grad_fn=<MulBackward0>) tensor(0.2500, dtype=torch.float64, grad_fn=<MseLossBackward0>)

✓
0s

[51] # call the optimizer

```
loss_comp.backward()
optimizer.step()
outputs = inputs * Inter_var
loss_comp = loss (outputs,targets) # loss compute
print( Inter_var,outputs,loss_comp)
```

tensor([0.3500], requires_grad=True) tensor([0.7000], dtype=torch.float64, grad_fn=<MulBackward0>) tensor(0.0900, dtype=torch.float64, grad_fn=<MseLossBackward0>)

✓
0s

[52] # One more round of the optimizer

```
loss_comp.backward()
optimizer.step()
outputs = inputs * Inter_var
loss_comp = loss (outputs,targets) # loss compute
print( Inter_var,outputs,loss_comp)
```

tensor([0.5100], requires_grad=True) tensor([1.0200], dtype=torch.float64, grad_fn=<MulBackward0>) tensor(0.0004, dtype=torch.float64, grad_fn=<MseLossBackward0>)